



Aula 4 (Parte 1) - Análise experimental e filtragem das medições

🕒 Criado em	7 de setembro de 2026 11:36
🏷️ Tags	

1. Primeiro: onde essa aula se encaixa?

Na aula anterior, o professor estava tentando responder:

“Qual é o tempo médio de execução de uma primitiva?”

Para isso, ele criou um pequeno programa que executava uma atribuição muitas vezes:

```
for i in range(n):  
    x = 1
```

E repetia esse experimento várias vezes.

A ideia era obter algo que representasse o tempo daquela operação.

Na aula anterior, o professor fazia uma busca pelo melhor ponto de corte usando o coeficiente de variação.

Agora ele parece ter mudado a estratégia.

Em vez de testar vários cortes arbitrariamente, ele começa calculando:

- média geral;
- desvio padrão geral;
- e então cria um limite usando a média + X vezes o desvio padrão.

A mudança parece pequena no código, mas **conceitualmente é importante**.

2. O objetivo continua sendo o mesmo

Antes de entrar no código, vamos lembrar o problema original.

Na disciplina, queremos conseguir estimar o tempo de um programa.

Para isso, chegamos à ideia de que podemos contar quantas primitivas existem:

```
Operações  
Comparações  
Atribuições
```

e depois associar um tempo médio a cada uma.

Por exemplo, conceitualmente:

```
10 operações  
20 comparações  
30 atribuições
```

poderiam ser utilizadas para estimar o tempo total.

Mas precisamos descobrir:

- | Quanto tempo demora, em média, uma operação?
- | Quanto tempo demora, em média, uma comparação?
- | Quanto tempo demora, em média, uma atribuição?

É aí que entra a **análise experimental**.

3. O professor começa com uma atribuição

O trecho que realmente interessa para o experimento é:

```
for i in range(n):  
    x = 1
```

Aqui:

```
x = 1
```

é uma atribuição.

O professor escolheu uma operação simples porque quer tentar medir seu custo experimentalmente.

Ele coloca essa atribuição dentro de um laço:

```
for i in range(n):  
    x = 1
```

Se:

```
n = 100000
```

então a atribuição é executada muitas vezes.

Isso é útil porque medir uma única atribuição seria extremamente difícil.

Imagine:

```
x = 1
```

demorar uma quantidade absurdamente pequena de tempo.

O cronômetro teria dificuldade para medir isso isoladamente de maneira útil.

Então fazemos:

```
1 atribuição
1 atribuição
1 atribuição
...
100000 atribuições
```

e medimos o conjunto.

4. Por que $n = 100000$?

Aqui:

```
n = 100000
```

Na aula anterior, tínhamos:

```
lista_n = [1000, 10000, 100000]
```

Ou seja, fazíamos o experimento com vários tamanhos.

Agora o professor fixou:

```
n = 100000
```

Isso significa que, nessa etapa específica, ele não está preocupado em estudar o crescimento com diferentes entradas.

Ele está preocupado em obter **uma boa medição experimental para esse caso**.

Essa diferença é importante.

Aula anterior

Uma preocupação era:

```
n = 1000
n = 10000
n = 100000
```

e observar como o tempo se comporta.

Aqui

Ele fixa:

```
n = 100000
```

e concentra-se em:

| “Como tratar corretamente as 100 medições que obtive?”

5. As 100 repetições continuam

Temos:

```
R = 100
```

e:

```
for r in range(R):
```

Portanto:

```
R = 100
```

significa:

| O experimento será executado 100 vezes.

O código mede cada execução:

```
tic = perf_counter()

for i in range(n):
    x = 1

toc = perf_counter()
tt.append(toc - tic)
```

Vamos visualizar:

```
EXECUÇÃO 1
100000 atribuições → tempo 1

EXECUÇÃO 2
100000 atribuições → tempo 2

EXECUÇÃO 3
100000 atribuições → tempo 3

...

EXECUÇÃO 100
100000 atribuições → tempo 100
```

No final:

```
tt
```

possui 100 tempos.

6. Por que os 100 tempos não são exatamente iguais?

Essa é uma questão fundamental para entender o restante da aula.

Você poderia pensar:

“Se o código é exatamente o mesmo e n é sempre 100000, não deveria dar exatamente o mesmo tempo?”

Não necessariamente.

O computador não está executando apenas o seu código.

Durante a execução podem existir outras atividades acontecendo:

```
Seu programa
+
Sistema operacional
+
Outros processos
+
Gerenciamento de memória
```

```
+  
Outras atividades do computador
```

Por isso podemos imaginar algo assim:

```
0.0031  
0.0032  
0.0031  
0.0033  
0.0032  
0.0048  
0.0031  
...
```

A maioria pode estar concentrada em determinada região, enquanto algumas medições podem ficar maiores.

7. É aqui que entra a estatística

O professor faz:

```
tt = np.array(tt)  
tt = np.sort(tt)
```

Primeiro transforma os dados em um array do NumPy.

Depois:

```
np.sort(tt)
```

ordena os tempos.

Imagine:

```
Antes:  
  
0.0032  
0.0048  
0.0031  
0.0033  
0.0030  
0.0065  
0.0032
```

Depois:

```
0.0030
0.0031
0.0032
0.0032
0.0033
0.0048
0.0065
```

Agora fica muito mais fácil observar os valores maiores e menores.

8. A primeira grande mudança em relação à aula anterior

Na aula anterior, o professor procurava diferentes valores de corte.

Agora ele faz:

```
media = np.mean(tt)
desvio = np.std(tt)
```

Ou seja, antes de filtrar qualquer coisa, ele calcula as estatísticas de **todas as medições**.

Temos:

```
media
```

e:

```
desvio
```

9. O que significa a média?

A média é simplesmente o valor central obtido somando os tempos e dividindo pela quantidade de medições.

Imagine:


```
10
11
10
9
10
```

A média seria:

```
10
```

No programa:

```
media = np.mean(tt)
```

Portanto:

`media` representa o tempo médio das 100 execuções antes de qualquer filtragem.

10. E o desvio padrão?

O desvio padrão tenta responder:

“Quanto os tempos estão espalhados em torno da média?”

Imagine:

```
10
10
10
10
10
```

Os valores são muito próximos.

Então:

```
desvio pequeno
```

Agora:

```
2
7
10
15
20
```

Os valores estão muito mais espalhados.

Então:

```
desvio grande
```

No código:

```
desvio = np.std(tt)
```

11. Agora vem a parte nova: X

O professor faz:

```
for x in [2, 3]:
```

Isso significa que ele vai testar dois valores:

```
x = 2
x = 3
```

E aqui é importante não confundir esse `x` com:

```
x = 1
```

da atribuição anterior.

São coisas diferentes.

No primeiro caso:

```
x = 1
```

`x` é uma variável do programa que está sendo medido.

Já aqui:

```
for x in [2, 3]:
```

`x` é uma variável usada pelo próprio programa de análise para representar um multiplicador.

Eu até chamaria mentalmente esse segundo `x` de:

```
multiplicador
```

para não confundir.

12. O que significa "média + X vezes o desvio"?

Aqui está provavelmente a parte mais importante da aula:

```
tcorte = media + x * desvio
```

O professor está criando um **tempo de corte**.

A ideia é:

```
tempo de corte =  
média + x × desvio
```

Como ele testa:

```
x = 2
```

teremos:

```
tempo de corte = média + 2 × desvio
```

Depois:

```
x = 3
```

teremos:

```
tempo de corte = média + 3 × desvio
```

13. Vamos colocar números fictícios

Suponha que o experimento tenha produzido:

```
média = 10  
desvio = 2
```

Quando:

```
x = 2
```

temos:

```
tcorte = 10 + 2 × 2  
tcorte = 14
```

Então o corte fica:

```
14
```

Agora:

```
x = 3
```

temos:

```
tcorte = 10 + 3 × 2  
tcorte = 16
```

Então:

```
x = 2 → corte = 14  
x = 3 → corte = 16
```

Perceba:

| Quanto maior o multiplicador, mais permissivo fica o filtro.

14. O que o filtro faz?

Depois:

```
tt_filtrado = tt[tt < tcorte]
```

Isso significa:

| Pegue somente os tempos menores que o tempo de corte.

Por exemplo:

Tempos :

```
9  
10  
10  
11  
12  
15  
20
```

Se:

```
tcorte = 14
```

ficam:

```
9
10
10
11
12
```

E saem:

```
15
20
```

15. Por que fazer isso?

Porque o professor está tentando eliminar valores que podem representar **medições anormalmente altas**.

Imagine que normalmente o programa demora:

```
10 ms
```

mas em uma execução o sistema operacional estava muito ocupado:

```
50 ms
```

Esse valor pode distorcer a análise.

Então queremos encontrar uma região mais representativa das execuções normais.

16. Mas por que testar 2 e 3?

Essa é uma escolha metodológica do professor.

Ele está comparando dois critérios:

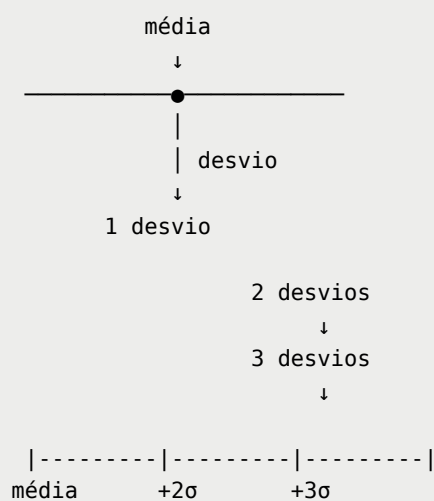
$\text{média} + 2 \times \text{desvio}$

e:

$\text{média} + 3 \times \text{desvio}$

O segundo é mais amplo.

Visualmente:



A ideia geral é:

+2 desvios
→ filtro mais rigoroso

+3 desvios
→ filtro mais permissivo

17. Depois ele recalcula tudo

Depois de filtrar:

```
media_filtrada = np.mean(tt_filtrado)
desvio_filtrado = np.std(tt_filtrado)
```

```
coef_variacao = desvio_filtrado / media_filtrada
```

Agora temos uma diferença importante.

Antes:

```
media  
desvio
```

eram calculados usando **todos os tempos**.

Agora:

```
media_filtrada  
desvio_filtrado
```

são calculados somente usando os tempos que passaram pelo filtro.

Então:

```
100 medições  
  ↓  
calcula média/desvio  
  ↓  
cria corte  
  ↓  
remove valores acima do corte  
  ↓  
medições filtradas  
  ↓  
calcula nova média/desvio
```

18. E volta o coeficiente de variação

Temos:

```
coef_variacao = desvio_filtrado / media_filtrada
```

Como vimos na aula anterior:

```
coeficiente de variação =
```


desvio padrão / média

Ele serve para avaliar a dispersão dos dados em relação à média.

Por exemplo:

$CV = 0.05$

significa uma variação relativa de aproximadamente:

5%

Enquanto:

$CV = 0.20$

corresponde aproximadamente a:

20%

19. Agora podemos comparar os dois filtros

O professor imprime:

```
print(f"x = {x}")
print(f"Tempo de corte = {tcorte}")
print(f"Média = {media_filtrada}")
print(f"Desvio = {desvio_filtrado}")
print(f"Coeficiente de variação = {coef_variacao}")
print(f"Quantidade de tempos = {len(tt_filtrado)}")
```

Então ele consegue observar algo como:

```

x = 2
Tempo de corte = ...
Média = ...
Desvio = ...
Coeficiente de variação = ...
Quantidade de tempos = ...

x = 3
Tempo de corte = ...
Média = ...
Desvio = ...
Coeficiente de variação = ...
Quantidade de tempos = ...

```

E isso permite comparar:

	x = 2 -----	x = 3 -----
corte	menor	maior
dados usados	menos	mais
média	...	...
desvio	...	...
CV	...	...

20. O que está acontecendo conceitualmente?

Essa aula está aprofundando uma ideia que já apareceu na anterior.

O professor está percebendo que:

Medir o tempo de execução não significa simplesmente pegar o primeiro número que o cronômetro retorna.

Existe todo um processo:

MEDIÇÃO EXPERIMENTAL

↓

Executar várias vezes

↓

Obter vários tempos

```
↓  
Analisar a distribuição  
↓  
Identificar valores discrepantes  
↓  
Filtrar  
↓  
Calcular novamente as estatísticas  
↓  
Obter uma estimativa mais confiável
```

Essa é provavelmente a **principal ideia da aula 4** nesse trecho.

21. Uma coisa MUITO importante: o professor não está analisando a complexidade ainda

Esse ponto é importante para não misturar as etapas.

Quando ele faz:

```
media = np.mean(tt)
```

não está calculando complexidade.

Quando faz:

```
desvio = np.std(tt)
```

também não.

Quando faz:

```
tcorte = media + x * desvio
```

também não.

Ele está **tratando os dados do experimento**.

A complexidade entra em outra etapa.

Podemos separar:

ETAPA 1 – EXPERIMENTO

Executar código

↓

medir tempo

↓

obter dados

ETAPA 2 – TRATAMENTO DOS DADOS

média

↓

desvio

↓

filtro

↓

média filtrada

ETAPA 3 – ANÁLISE DO PROGRAMA

contar primitivas

↓

obter função de tempo

↓

analisar crescimento

↓

complexidade

Isso vai ajudar bastante quando os códigos começarem a ficar maiores.

22. E aquele `plt` ?

Perceba uma coisa curiosa.

No código que você mandou, temos:

```
from matplotlib import pyplot as plt
```

mas não temos:

```
plt.hist(...)  
plt.show()
```

como na atividade anterior.

Então, **neste trecho específico**, o `matplotlib` não está sendo utilizado.

Provavelmente o professor simplesmente manteve o import do código anterior ou pretendia utilizá-lo em outro momento.

Isso não interfere na análise principal.

23. E aquele último `print` ?

No final temos novamente:

```
print(f"n = {n}")  
print(f"x = {x}")  
print(f"Tempo de corte = {tcorte}")  
print(f"Média = {media_filtrada}")  
print(f"Desvio = {desvio_filtrado}")  
print(f"Coeficiente de variação = {coef_variacao}")  
print(f"Quantidade de tempos = {len(tt_filtrado)}")
```

Tem uma peculiaridade aqui.

Como esse `print` está **fora** do:

```
for x in [2, 3]:
```

ele vai imprimir os valores referentes à **última execução do laço**, ou seja:

```
x = 3
```

e seus respectivos resultados.

Então esses últimos `print`s são meio redundantes.

O resultado de `x = 2` já foi impresso dentro do laço, e o resultado de `x = 3` também.

Portanto, esse último bloco não acrescenta uma nova análise; ele apenas repete os valores finais.

24. Comparando com a aula anterior

Agora fica bem mais fácil enxergar a evolução.

Aula anterior

O professor fazia algo conceitualmente parecido com:

```
100 medições
↓
ordenar
↓
testar vários cortes
↓
filtrar
↓
calcular média
↓
calcular desvio
↓
calcular CV
↓
parar quando CV < 15%
```

Aula 4

Agora:

```
100 medições
↓
ordenar
↓
calcular média geral
↓
calcular desvio geral
↓
testar  $X = 2$ 
↓
corte = média + 2 × desvio
↓
filtrar
↓
calcular estatísticas
↓
testar  $X = 3$ 
```

```
↓  
corte = média + 3 × desvio  
↓  
filtrar  
↓  
calcular estatísticas  
↓  
comparar
```

Então sim:

é uma continuação direta da atividade anterior, mas com uma estratégia diferente para definir o corte.

25. O que eu acho que você precisa entender dessa aula

Não precisa decorar cada linha do NumPy agora.

O importante é entender o **raciocínio por trás do código**.

Conceito 1 — Medição experimental

Executamos o programa e medimos o tempo real.

Conceito 2 — Repetição

Executamos várias vezes porque uma única medição não é confiável.

Conceito 3 — Variabilidade

Mesmo executando o mesmo código, os tempos podem variar.

Conceito 4 — Média

Representa o comportamento médio das medições.

Conceito 5 — Desvio padrão

Mostra o quanto os tempos estão dispersos.

Conceito 6 — Coeficiente de variação

Relaciona a dispersão com a média.

Conceito 7 — Tempo de corte

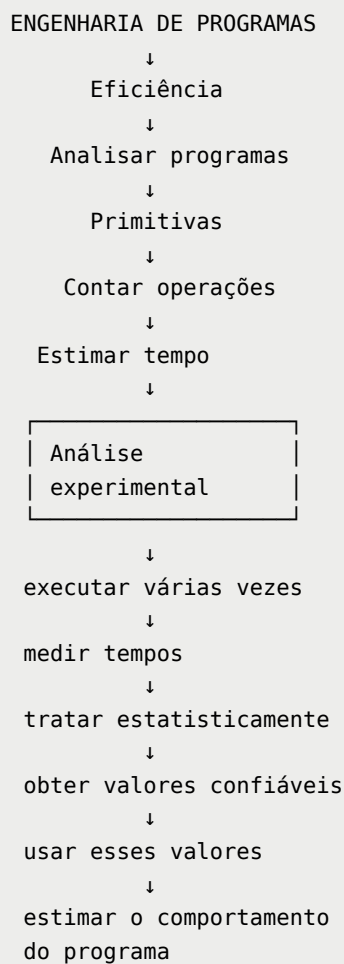
É um limite utilizado para eliminar valores considerados muito altos/discrepantes.

Conceito 8 — Média + $X \times$ desvio

É a estratégia usada nessa versão para definir esse limite.

26. A conexão com o que já estudamos

E aqui está a parte que eu acho mais importante para você acompanhar o professor daqui para frente:



Então **essa aula não está abandonando o que estudamos anteriormente.**

Ela está preenchendo uma lacuna:

"Beleza, sabemos que precisamos de um tempo médio para as primitivas.
Mas como conseguimos uma medição experimental confiável desse tempo?"

E esse código é uma tentativa de responder exatamente isso.
